

ENCM 511: Embedded System Interfacing
Project

Report

Group 6

Chloe Fulbrook - 30210975

Ethan Lee - 30213225

Devon Calvin - 30204532

Submitted on December 1st, 2025

Functions

LED Module Functions

`void v_blink_callback_count(TimerHandle_t xTimer)`

```
void v_blink_callback_count(TimerHandle_t xTimer){
    // this timer callback is responsible for creating the 1s blink rates
    of LEDs 1 and 2
    led2_blink_phase ^= 1; // toggles blinking phase of LED2. this is
    used in the LED2 state machine
    if(led1_blink_en){
        // if LED1 is enabled to blink, toggles the pin at 1s intervals
        LED1 ^= 1;
    }
}
```

It is a callback function for the software timer, `led_blink_timer`. When this function executes it XORs `led2_blink_phase` which will cause LED2 to blink on and off at variable intensity. If `led1_blink_en` is high then LED 1 also blinks at full brightness.

`void v_blink_callback_alternating(TimerHandle_t xTimer)`

```
void v_blink_callback_alternating(TimerHandle_t xTimer){
    // this timer callback is responsible for LEDs 0 and 1 blinking
    rapidly in an alternating fashion
    // each LED state is toggled, but are alternating because upon
    starting the timer, they are in opposite states
    LED0 ^= 1;
    LED1 ^= 1;
}
```

It is a callback function for the software timer, `led_alternate_timer`. When this function executes it XORs the current state of LED0 and LED1 with 1. Before the timer is turned on, LED0 and LED1 are set to have opposite states. When the timer executes, it will cause a blinking effect.

`void __attribute__((interrupt, no_auto_psv)) _T2Interrupt(void)`

```
void __attribute__((interrupt, no_auto_psv)) _T2Interrupt(void){
    /*
    * the T2 interrupt handles the small state machine on LED2. This LED
    must be able to pulse, blink,
    * and be steady on/off, as well as maintain a variable intensity as
    given by the ADC potentiometer.
    */
}
```

```

    * Because of the multiple modes, it makes sense to implement the LED
as a state machine.
    *
    * We use a hardware timer for this to ensure responsiveness to the
PWM changes.
    */

 BaseType_t xHigherPriorityTaskWoken = pdFALSE;

 TMR2 = 0;                // reset the timer start count
 IFS0bits.T2IF = 0;       // clear the interrupt flag
 uint16_t duty;           // an intermediate/internal variable to hold
the current duty cycle

 switch(led2_mode){
     // switch on the mode of the LED2 state machine
     case LED_MODE_OFF:
         // if the led should be off, disable output and set the duty
to 0

         led2_output_en = 0;
         duty = 0;
         break;
     case LED_MODE_ON:
         // if the led should be constantly on, enable the output and
adjust the variable duty according to the ADC reading
         led2_output_en = 1;
         xSemaphoreTakeFromISR(led2_sem, &xHigherPriorityTaskWoken); //
semaphore protected bc duty gets modified in the adc isr also
         duty = led2_variable_duty;
         xSemaphoreGiveFromISR(led2_sem, &xHigherPriorityTaskWoken);
         break;
     case LED_MODE_BLINK:
         // if the led should be blinking, set the variably duty
according to the adc
         xSemaphoreTakeFromISR(led2_sem, &xHigherPriorityTaskWoken);
         duty = led2_variable_duty;
         xSemaphoreGiveFromISR(led2_sem, &xHigherPriorityTaskWoken);
         led2_output_en = led2_blink_phase; // and enable the output
based on the phase of the blinking
         break;

```

```

    case LED_MODE_PULSE:
        // if the led should be pulsing (as in the wait state of the
fsm)

        led2_output_en = 1; // enable output
        if(decrease){
            // if the decreasing flag is on, we decrease the pwm on
time by pulse_step_size to decrease duty
            PWM_TON -= PULSE_STEP_SIZE;
            if(PWM_TON <= PULSE_STEP_SIZE){
                // now if the next step will wrap the variable, change
the decreas
                    ing flag to increasing
                    decrease = 0;
            }
        }
        if(!decrease){
            // if the decreasing flag is off, increase the pwm on time
by pulse_step_size to increase duty
            PWM_TON += PULSE_STEP_SIZE;
            if(PWM_TON >= PWM_PER - PULSE_STEP_SIZE){
                // if the next step will go past the period of the
pwm, swap to be increasing
                    decrease = 1;
            }
        }
        duty = PWM_TON; // assign the on time of the pwm to the duty
for later use
        break;
    } // end switch(led2_mode)

    if(led2_output_en){
        // if the output is enabled, create the pwm based on the earlier
assigned duty
        if(pwm_on){
            // if the pwm_on is high, do one phase of the pwm
            LED2 = 0; // turn led2 off
            PR2 = PWM_PER - duty; // set the timer count to the
period-duty
            pwm_on = 0; // switch the flag
        }
    }

```

```

        else{
            LED2 = 1;                // turn led2 on
            PR2 = duty;              // set timer count to the duty cycle
            pwm_on = 1;              // switch the flag
        }
    }
    else{
        // if the led2 output is not enabled, force LED2 off
        LED2 = 0;
    }

    if(xHigherPriorityTaskWoken) {
        portYIELD(); // triggers context switch if a higher priority task
is woken
    }
}

```

This interrupt handles the logic for LED 2's behavior. There are four behaviors covered by the case statements. LED_MODE_OFF, which keeps LED 2 off and sets the duty cycle to 0. LED_MODE_ON, which enables LED 2 to output at some variable intensity. One important note is that that duty, the variable used to control the intensity, is protected by a semaphore. This is because the value it is assigned happens in the ADC interrupt and without the semaphore, there could be a race condition. Another mode is LED_MODE_BLINK, where the semaphore protected duty variable is assigned the duty cycle from the ADC. In this mode, led2_output_en is also toggled to produce the blinking effect at the variable intensity. The last mode is LED_MODE_PULSE where led2_output_en is enabled and the duty cycle is varied from 0 to 100% repeatedly, which produces the pulsing effect on LED 2. If during this ISR, a higher priority task is awoken, portYIELD is executed which context switches the interrupted task to start executing the higher priority task.

UART Module Functions

*void InitUART2, void Disp2String(char *str), void XmitUART2(char CharNum, unsigned int repeatNo), void Disp2Dec(uint16_t Dec_num), void int_to_str_dec(uint16_t dec, char *str)*

These are the functions used in the UART file. The functionality of these functions have not changed from the previous labs and have already been described in those labs.

```

void __attribute__((interrupt, no_auto_psv)) _U2RXInterrupt(void)
void __attribute__((interrupt, no_auto_psv)) _U2RXInterrupt(void) {
    /*
     * This UART RX ISR is used to collect and queue user keyboard inputs
     */

    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    uint8_t c;        // variable for char storage

    c = U2RXREG;      // read the UART2 receive register for the char

    xQueueSendFromISR(uart_rx_queue, &c, &xHigherPriorityTaskWoken);    //
    send to queue for use in main

    IFS1bits.U2RXIF = 0;    // clear interrupt flag
    RXFlag = 1;             // set rx flag (used in other uart functions,
    like recvuart)

    if(xHigherPriorityTaskWoken){
        portYIELD();        // context switch
    }
}

```

The UART receive interrupt is used to receive the character from the register that was transmitted from the keyboard. The character it reads is sent to the main task the the uart_rx_queue. To jump to the higher priority task that becomes unblocked after sending the character to the queue, portYIELD is used to context switch from the interrupted task directly to the ready higher priority task after the ISR executes.

ADC Module Functions

```

void ADC1_init(void) {

    AD1CON2bits.PVCFG = 0;    // positive voltage reference to AVDD
    AD1CON2bits.NVCFG0 = 0;   // negative voltage reference to AVSS

    AD1CHSbits.CH0NA = 0;     // neg ref to Vss for channel a
    AD1CHSbits.CH0SA = 5;     // pos ref to AN5 (pin7) for channel a
}

```

```

AD1CON3bits.ADCS = 13;          // Tad = 14 * Tcyc = 291ns

// DMAEN, ASEN are 0

AD1CON2bits.SMPI = 0;          // interrupt after one sample

AD1CON3bits.ADRC = 0;          // use system clock

AD1CON1bits.SSRC = 2;          // set to use tmr3 trigger

AD1CON3bits.SAMC = 26;         // 26TAD sample time
AD1CON1bits.FORM = 0;          // absolute decimal result
AD1CON1bits.MODE12 = 0;        // 10 bit mode

IEC0bits.AD1IE = 1;           // enable interrupts
IFS0bits.AD1IF = 0;           // clear flag

AD1CON1bits.ASAM = 1;          // auto sample (or can manually set)

}

```

This function initializes the ADC. The positive reference is set to VDD, the negative is set to AVSS. The negative reference for channel a is Vss and the positive reference is AN5 or pin 7. The conversion clock was calculated to be 291ns so that it is greater than the minimum Tad 278 ns. The ADC is set to execute every time Timer 3 is triggered. The sample time was calculated to be 26Tad from (RS + RIC + RSS)(CHOLD). The results are stored in the buffer as absolute decimal results. It's also configured in 10-bit mode and has interrupts enabled. Auto sample is also enabled.

```
void __attribute__((interrupt, no_auto_psv)) _ADC1Interrupt(void)
```

```

void __attribute__((interrupt, no_auto_psv)) _ADC1Interrupt(void) {
    /*
     * The adc interrupt handles the potentiometer-set variable brightness
     of LED2.
     * For this application, we are not polling the done bit, rather
     relying on the ISR.
     * The adc is set according to the running of T3, so it runs
     periodically to allow for responsiveness
     */
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

```

```

    raw_adc_result = ADC1BUF0;          // store the result in the adc buffer
as the raw result

                                         // necessary because raw result is
used to display the adc reading on the screen

    adc_result = raw_adc_result;        // set the adc result as the raw to
help with clipping

    // here we're clipping the adc result to avoid overflowing or breaking
the pwm
    if(adc_result >= 1023) adc_result = 1022;
    if(adc_result <= 0) adc_result = 1;

    // transform the adc result into essentially a duty cycle, clipping if
necessary
    uint32_t temp = ((uint32_t)adc_result * PWM_PER) / 1023U;
    if(temp > PWM_PER) temp = PWM_PER; // Essentially protects a really
large number from being assigned to the duty cycle
    if(temp <= 1) temp = 1

    // we protect the variable duty with a semaphore to avoid races when
both the adc may be modifying the variable
    // and the led2 state machine may be reading it
    xSemaphoreTakeFromISR(led2_sem, &xHigherPriorityTaskWoken);
    led2_variable_duty = (uint16_t)temp;
    led2_intensity = led2_variable_duty * 100 / 400; // Intensity as a
percentage
    xSemaphoreGiveFromISR(led2_sem, &xHigherPriorityTaskWoken);

    IFS0bits.AD1IF = 0; // clear the adc interrupt flag

    if(xHigherPriorityTaskWoken) {
        portYIELD(); // triggers context switch if a higher priority task
is woken
    }
}

```

In this interrupt the ADC conversion has finished and the result is taken from the ADC buffer and assigned to raw_adc_result. This result is then assigned to adc_result where it is then clipped between 1022 and 1. This prevents overflows and glitches from occurring with the PWM. The

result is then converted into a duty cycle and assigned to a temporary 32 bit long type temp variable. The value of the duty cycle is then clipped if necessary so that it doesn't exceed the maximum duty cycle period that we set and doesn't go below 1. The duty cycle variable is then type casted back to 16 bits and assigned to led2_intensity. This intensity variable is used as described in the T2interrupt. At the very end, the ADC interrupt flag is cleared. Also, similar to the Timer 2 interrupt, portYIELD is used to context switch if a higher priority task was unblocked during the interrupt.

countdown module functions

void v_end_callback(TimerHandle_t xTimer)

```
void v_end_callback(TimerHandle_t xTimer){
    // this timer callback runs once the 5 seconds of timeout have elapsed
after the completion of the countdown
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // the callback notifies the main state task that the timer has
elapsed, and the main state deals with the transition
    xTaskNotifyFromISR(main_state_task_handler, (1<<NOTIFY_END_TIMEOUT),
eSetBits, &xHigherPriorityTaskWoken);
    if(xHigherPriorityTaskWoken){ // will always be true
        portYIELD();
    }
}
```

It is a callback function for the software timer end_timer. This is executed after the 5 second timer and it sends the NOTIFY_END_TIMEOUT notification to the main task. This notification is used to signal that 5 seconds have passed since the countdown timer has finished, meaning the main task can transition from STATE_END to STATE_WAIT. Additionally, since the callback functions run similarly to interrupts, portYIELD is used to jump to the higher priority task that was unblocked during its execution.

Global Variables		
Name	Shared Between	Purpose
raw_adc_result	ADC file	Reads the voltage from the potentiometer and stores the ADC value
	v_countdown_task	Stores a copy of this value within the struct type msg for the display task
led2_intensity	ADC file	Reads the voltage from the potentiometer and

	v_countdown_task	stores the ADC value Stores a copy of this value within the struct type msg for the display task
time_seconds	v_main_state_task v_countdown_task	Holds the timer that the user input in seconds Used to track if the task should execute, a copy is converted into minutes and seconds in ASCII, tracks when it equals 0
countdown_running	v_main_state_task v_countdown_task	Toggle this flag high or low depending on if PB3 is pressed during count down Used as 1 of the signals checked for if the task should execute. It is also used to sync taskdelayuntil when resuming.
display_mode	v_main_state_task v_countdown_task	Swaps between assigning it DISP_MODE_TIME and DISP_MODE_FULL on i keyboard press Used in a conditional statement that sets a flag within msg struct type for either displaying the timer only or timer with ADC reading and LED 2 intensity.
PWM_TON	v_main_state_task LED_module	Assigned to PR2 on initialization to pulse LED2 Value is incremented and decremented, changing the duty cycle and thus brightness of LED. This creates the pulsing effect.
PWM_PER	ADC module LED_module	Used in led2_intensity calculation. Defines the PWM period, used to help calculate PR2 to produce different brightnesses
led2_blink_phase	v_main_state_task LED_module	Sets the blink phase of LED2 to 1 Using callback function v_blink_callback_count and led2_mode case statement, causes LED2 to blink.
led2_mode	v_main_state_task	Sets the variable equal to case to produce appropriate LED behavior in each part of the FSM.

	LED_module	Used in case statement to choose which behavior the LED should have
led1_blink_en	v_main_state_task	Sets variable to 1 to enable LED 1 to blink
	LED_module	Used in conditional statements to decide whether to toggle led 1.
led2_variable_duty	ADC Module	Assigns the variable with the duty cycle calculated from the ADC reading
	LED Module	Assigns the value of this variable to the duty cycle of led2 while it is on or blinking
time_entry	Countdown module	Initializes time entry with 0's
	v_main_state_task	Time_entry holds the characters of the time the user input
time_ready	Countdown module	Initialized as zero
	v_main_state_task	Used as a flag to check if the user has input an acceptable number
time_entry_index	Countdown module	Initialized as zero
	v_main_state_task	Used to index time_entry so that it contains the right number in the appropriate cell. Also used to check if the user has input 4 numbers for the time.

General Code Explanation

For the project, the codebase is separated into a number of different files and modules. This helps with the readability and organization of the project.

main.c

This is the main file for the project, including main() and the main state task for the overall finite state machine. It begins with all of the necessary #pragmas for setup and configuration, and includes the necessary headers. We then declare the enum type for the overall state machine, and default the current state to STATE_WAIT. We also set up the task handle for the main state task, which will be used to send notifications to this task from others.

Two tasks were given in the FreeRTOS starter package, and have been slightly modified from how they were given. The `vApplicationIdleHook()` has been modified to include the `Idle()` call to force the CPU to idle when other tasks are not running. There is also the `vApplicationStackOverflowHook()`, which runs when a stack overflow has been detected. This disables interrupts, and will print the name of the task that caused the overflow.

The file continues to the function `prvHardwareSetup()`, which is used to call the initializer functions for microcontroller peripherals. This includes initializing the UART, ADC, timers, and IOs, and their specific functions will be described in more detail following.

The `main()` function also exists in this file, and is largely used for setup. Here, we call the `prvHardwareSetup()` function to initialize the IOs, and set pin 7 (RB3) for the ADC input from the potentiometer. The tasks used in the application are then created, including for the button, the main states, countdown, and display. The priorities of these are described in more detail later in this report.

Next, we create the software timers that are used. First is the LED blink timer, which is used to create the blinking of LEDs 1 and 2. It runs at `BLINK_RATE`, which is defined as 1s as per the given requirements. It is also set to auto reload so the blinking is continuous. This timer is immediately started. The next software timer created is the `led_alterate_timer`, which is used to create the rapid alternating blinking of LEDs 0 and 1 at the end of the countdown. It runs at `ALT_BLINK_RATE`, or 200ms, and will again auto reload to ensure the blinking is continuous while the timer is on. The last timer created is the `end_timer`, which allows for the 5s timeout after the countdown has completed. It runs at `END_TIMEOUT`, which is 5s, and is a one-shot timer, as that is all that is required for this part.

Following the timer set up, we create the semaphores and mutexes used in the application to protect shared resources. A mutex is used to protect the UART, helping to avoid garbled or nonsense output if two tasks try to write too close to each other. Mutexes are also used for the variable duty of LED2 and the `time_seconds`. This is because these variables are written to and read from multiple tasks, and we are attempting to avoid race conditions. A binary semaphore is created for the buttons, and is used to notify the button task of IOC events. It is important to notice that because we are using this semaphore for notification and synchronization as opposed to resource protection, a binary semaphore is a better choice than a mutex.

The function continues to create some queues to aid with inter-task communication. The `processed_button_queue` is used to send debounced and processed button events from the button task to main, and is created with a size of 10 `processed_event_t`. Similarly, the `display_queue` sends display commands from the countdown task to the display task, and has size of 10 `display_msg_t`. The `uart_rx_queue` is used to send user-inputted chars from the UART RX ISR to

the main state task for handling. This queue is created to be larger than the others as it may run faster, and is set to hold 32 `uint8_t`.

`main()` continues to display the startup message “Press PB1 to begin!”, and that UART write is protected by the semaphore. We then configure the hardware timers that are used in the application. Timer2 is used for the LED2 state machine, and here the TMR2 is set to 0 as the start count, and is initialized to PWM_TON as the stop count. The timer is then turned on to begin the pulsing of LED2. Timer3 is used as the ADC conversion trigger, and is set to continuously run at ADC_SAMPLE_TRIGGER (200 ticks). The Timer3 and the ADC are then turned on, and thereafter the timer will automatically trigger the ADC to convert when the count is up. Finally, we start the scheduler, allowing the FreeRTOS tasks to begin running.

The last part of *main.c* is `v_main_state_task()`, which holds the overall state machine for the system. For our implementation, all of our state transitions occur in this task, to help avoid spaghetti logic and help with containment. The task begins by creating variables to store the information coming from the queues and task notifications. The task then enters the `for(;;)` loop, and checks to see if anything was received from either the processed button queue, or from the notifications, and sets a respective flag. If neither of these events were received, the task does a short delay to yield control of the CPU and allow lower priority tasks to run.

The task then switches over the `curr_state` to implement the state machine. In `STATE_WAIT`, we expect there to be a start message and for LED2 to be pulsing smoothly from dim to bright. For this, we ensure that the LED2 is set to pulse mode, and then check if a button event was received. If the processed event was a click, and the event was on PB1, a state transition occurs. This is based on the structure type that is defined for processed events, where the type and button mask are passed as part of the queue. On the transition edge, we set the new `curr_state` to `STATE_INPUT`, and set the `time_ready` flag to 0 to allow for the user to input time in the next state. We display the time entry message, ensuring that the UART display is protected by the mutex.

In `STATE_INPUT`, we expect that the user will be able to enter the desired countdown time via the keyboard, and the display should update as they enter numbers. All LEDs should be off at this point. Transition out of the state should occur on a combination click of PB2 and PB3, and the time entry should reset, allowing the user to put in a different time. To implement this, upon entering the case, we set LED2 to its off state, and proceed to check if a button event was received.

If a combination click was received on PBs 2 and 3, we check the `time_ready` flag to see if the user has entered a valid time. If yes, the state will progress to `STATE_COUNT`, displaying the countdown started message (with mutex protection), set the `countdown_running` flag, and start

the blinking of LEDs 1 and 2. To make them blink in phase, we force them to start blinking in the same phase, and enable the blinking. Otherwise, if the time is not ready, the state loops back to `STATE_INPUT`, and we reset the time entry. The index is set back to 0, and the buffer is cleared to all 0 as well. We also reset the `uart_rx_queue` to ensure no artifacts are carried over, and we display a mutex protected message to tell the user that the time was not ready, and allow them to re-input. Generally, this condition will only be true if the user has clicked PB2 and 3 before entering four digits.

If a long combination click was received on PBs 2 and 3, the time entry should be reset. The state loops back to `STATE_INPUT`, and we clear the ready flag, entry index, and entry buffer to all 0. We again reset the queue to avoid artifacts, and re-print the time entry message.

Next, we deal with the user UART input for the time. If the `time_ready` flag is not set, the system is still accepting user input. We check for items in the queue, using a `portMAX_DELAY` wait time, as this task cannot continue without user input. The received byte is stored as a char, and we check to ensure that it is a digit (i.e. within the range of '0' to '9'). If it is not a valid digit, the character is consumed by the queue and is ignored. The input char is placed at the current `time_entry_index` of the `time_entry` buffer, and it is printed to its respective location on the screen. Then the `time_entry_index` is incremented, and we wait for another character to be entered by the user. After printing the second character to the screen, we use an escape code to skip one space of the cursor, to preserve the ':' that is printed. This continues until all places in the `time_entry` buffer are filled, at which point we reset the index and set the `time_ready` flag high. This signals that four valid characters have been entered, and the system can continue.

Now if the time becomes ready, the `time_entry_buffer` holding the individual digits is converted to a time in seconds. It is worth noting that `time_seconds` is protected by a mutex, as it is written and read in different locations. Then, we check to see if the entered time is more than 99:59, as this is effectively the cap for how our display prints. If >99:59, the time in the display task that is limited to the format mm:ss, it will overflow and cause display issues. For these cases, our group has decided to cap the countdown to count from 99:59.

`STATE_COUNT` is responsible for the countdown and associated LED behaviour, as well as accepting UART input characters and potentiometer changes. Clicking PB3 in this state should pause and resume the countdown. Inputting a 'b' over UART should toggle LED2 from blinking to solidly on. Inputting 'i' should toggle the display from showing only the time left to having the time left, ADC reading, and LED intensity. The brightness of LED2 should vary based on the potentiometer input in both the blinking and steady-on states. Transition out of this state should occur either on timer elapse, or a long click of PB3 to abort the timer.

To implement this, we first check if a button event was received by the task by checking `got_button`. If the event was a click on PB3, we are either pausing or resuming the timer, so we XOR to toggle `countdown_running`. If `countdown_running` is 0, the countdown has been paused, so we print that message to the screen. We then turn off the blinking timer if it is active to pause the blinking. Now if `countdown_running` is 1, the countdown has been resumed, and we print the resumed message to the screen. We also restart the blinking timer if it is inactive in order to restart the blinking.

Now if the button event was a long click on PB3, we want to abort the countdown. For this, we disable LED1 from blinking, turn LED2 to solid on, and force LEDs 0 and 1 to opposite states to allow them to do the alternating blink required in the end state. We start the alternating timer for the LEDs, and also start the `end_timer` that provides the 5s timeout of the alternating blink. Next, `time_seconds` is returned back to 0, and we set `countdown_running` to 0 to disable the countdown. The countdown aborted message is printed to the screen, and we transition to `STATE_END`.

If a notification has arrived to the main state task, and the notification is that the countdown has finished, the same LED configurations and timer set up occur as above with the long click on PB3. The only difference is that we print the “countdown finished” message before transitioning states instead of the “countdown aborted” message. Note that we do not need to force `time_seconds` to 0, as it has already reached 0 during the countdown.

Still in the counting state, we finally check the `uart_rx_queue` for user keyboard input. This queue receive works very similarly to the one for the time entry, except instead of checking if the entered char was a digit between 0 and 9, we check for the input of ‘b’ and ‘i’. If the user enters ‘b’, we toggle the mode of LED2 between blinking and solidly on. If ‘i’ is entered, we toggle the display mode between `DISP_MODE_FULL` and `DISP_MODE_TIME`. Because we only check the conditions of these two characters, any other input is consumed by the queue and is ignored. Finally, we reset the queue to avoid erroneous inputs or artifacts.

The final part of the state machine deals with `STATE_END`. Here, LEDs 0 and 1 blink rapidly in an alternating fashion and LED2 remains solidly on, with variable brightness. Transition out of this state should occur when the 5 second timeout has elapsed. The LED behaviour is set on the edge of the state transitions into `STATE_END`. If main is notified that the end timeout has elapsed, we turn LEDs 0 and 1 off, and stop the alternating timer if it is active. We then print the startup message again, and transition back to `STATE_WAIT`.

adc.c

This file contains the functions and ISRs necessary to implement the variable brightness on LED2 via the ADC. It begins by including its header file, and declaring volatile variables for the

raw adc result, which is used to display the adc reading on the screen, the clipped adc result, and for the LED2 intensity.

The first function is the `ADC1_init()` function. Here, we configure the ADC for use in the application. We set the positive and negative voltage references to `Vdd` and `Vss`, respectively. We use channel A, so we set the channel A negative reference to `Vss` and the positive reference to `AN5` (pin 7), where the potentiometer is connected. `ADCS` is set to 13 based on calculations from the datasheet. `SMPI` is set to 0 so that the ADC causes an interrupt after each collected sample, to help with responsiveness. `ADRC` is set to 0 to use the system clock as the ADC's clock source. `SSRC` is set to 2 to use timer 3 as the conversion trigger. Here, essentially the ADC is told to convert every time that timer 3 reaches its peak count. In our application, this occurs every 200 ticks. We use the timer 3 as the trigger to help with responsiveness in the system, as well as to avoid having to continuously call ADC functions. `SAMC`, the auto sample time select, is set to 26, again based on calculations from the datasheet. `FORM` is set to 0 to get the ADC result as absolute decimal. We also configure the ADC to 10 bit mode by setting `MODE12` to 0. Then we enable the interrupt and clear the flag, and set the ADC to auto sample.

The other important part of this file is the ADC interrupt service routine. For this project, we have decided to utilize the ADC interrupt as opposed to polling the `DONE` bit. Based on the earlier described configuration, upon the ADC conversion being triggered by timer 3, one sample is collected before the interrupt occurs. This sample is, by default, stored in `ADC1BUF0`. We read this register, and store the result in `raw_adc_result` to be used with the display of the ADC result. The raw result is also stored in `adc_result`, but `adc_result` must be clipped to avoid overflowing or breaking the PWM control. If the result is 0, we clip it to 1, and if it is 1023 or larger, we clip it to 1022. Next, the `adc_result` is converted to what is essentially a duty cycle, and clamp it to be within the range of the `PWM_PER`. Then we assign the temporary variable to the `led2_variable_duty` and calculate the `led2_intensity` by converting the duty to a percentage. Note that `led2_variable_duty` is protected by a mutex, as it is read and written in multiple functions and tasks. We then clear the ADC interrupt flag, and yield the CPU control if a higher priority task is woken.

adc.h

This is the header file accompanying the `adc.c` file. Here, we declare the adc variables that are used in multiple files as externs. It also includes the function prototypes for the ADC functions.

button_module.c

This file is the source file that contains the task and ISR required for the buttons to function. It begins by including its header file, and declares a variable to track active presses. It also holds the declarations of the `processed_button_queue` and the `button_sem`.

The file contains the IOC ISR. There is only one combined IOC available, so all change interrupts enter this ISR. Being configured to interrupt on both positive and negative edges of all of the button pins, this service routine is entered frequently. Upon an interrupt being triggered, we check which pin it was on to clear the specific pin's flag, and we give the `button_sem` from the ISR. We then clear the global flag, and allow the CPU to yield if a higher priority task is woken.

The main task in this file is `v_button_task()`. This task is responsible for both the debouncing and the combo and long press detection on the buttons. We begin by reading the initial states of the buttons, and declare a `last_stable_state` for comparisons. Inside of the task `for(;;)` loop, we take the `button_sem` given from the ISR, and proceed to wait for `DEBOUNCE_DELAY` (50ms) to allow the buttons to stabilize before reading their states. We now read the debounced states of the buttons, and store them as a mask in `curr_button`. It is worth noting that `curr_button` holds the logical state of the button, with 0 being pressed and 1 being released. For us, we found it easier to track active presses as opposed to logical states, so we convert the `curr_button` to `pressed_mask`, which has a 1 if the button is pressed. Now we check if the `pressed_mask` is different from `last_stable_state`, and only do button logic if something has changed. From here, we update the `last_stable_state` to the `pressed_mask` for future comparison. If the `pressed_mask` is not all zeros, at least one button is being pressed, so an active press is occurring. We update the structure type of `active_press` to hold the current pressed mask, the timestamp of the press using `xTaskGetTickCount()`, and `.active` that shows that the button is being tracked. Now, if the `pressed_mask` is zero, and `.active` is 1, no buttons are currently being pressed, but we were just tracking a press. This means that the buttons have been released, so some sort of click has been completed. We calculate the duration of the press by subtracting the timestamp of the press from that of the release. Next, a `processed_event_t` `pevt` is created to send the button event over the queue to the main state task. The button mask of `pevt` is set to the mask of the active press, and we collect a timestamp. We loop over the mask to determine how many buttons are pressed in order to do combo detection. If `num_pressed` is one, only one button was pressed, and we set `pevt.type` to the appropriate mode based on the duration of the click. The same logic applies to if `num_buttons > 1`, where we set the type based on the duration of the click. From there, we send the processed event over the `processed_button_queue` to the main state task, and clear the active flag in the `active_press`.

button_module.h

This is the header file associated with the button module. It begins by including the necessary headers, then proceeds to define macros. These macros are the `DEBOUNCE_DELAY`, which is defined to be 50ms in ticks, `NUM_BUTTONS`, which is 3, and `LONG_PRESS_DUR`, which is 750ms in ticks. We also define macros for the pushbuttons, and masks for the required button combos. Next, a structure type is defined for `button_event_t` which is used for the `active_press` described above, and includes a mask, active flag, and timestamp. An enum type is used for

event_type_t which holds the type of the button event, being click, long click, combo click, and long combo click. Another structure type is used for processed_event_t to send events to the main state task, and includes a timestamp, mask, and event type. The external handles for the button_sem and processed_event_queue are included here as well. Finally, a function prototype for the v_button_task() is included as well.

countdown_module.c

This file holds the module that controls the countdown and associated display. It begins by including its header file, and proceeds to declare the necessary variables. The time_entry array is defined to be the width of the NUM_ENTRY (in our case, 4), and is initialized to all zero. This gets updated as the user enters the time for the countdown. The time_ready flag and time_entry_index are initialized to 0, along with time_seconds. The countdown_running flag is also initialized to 0, as the countdown does not start running immediately. Time_string is a buffer initialized to the width of NUM_ENTRY + 2 to ensure extra space for the null terminator. The declarations for the end_timer, display_queue, uart_sem, and time_sem are included also, and the display mode is defaulted to be the DISP_MODE_TIME.

The next inclusion is the v_end_callback(), which is the timer callback for the 5 second timeout upon timer elapse or abort. When this callback runs, it notifies the main state task that the end timeout has occurred using xTaskNotifyFromISR, and notifies with the NOTIFY_END_TIMEOUT from the enum type main_notify_t. The timer yields if a higher priority task is woken.

The file continues to v_countdown_task(), which is responsible for the actual timer countdown of the application. A xLastWakeTime is collected using xTaskGetTickCount(), and a display_msg_t type msg to be queued to the display task. Within the for(;;) loop of the task, we check to see if the countdown is running, and that the time_seconds is more than zero. If true, we xTaskDelayUntil() for 1s to create the timer and make it periodic and as accurate as possible. Then, if countdown_running becomes 0 during that wait, we get a new wake time to resynchronize the timer anchor, and help stop the print of an additional time after pause or abort of the timer. We then proceed to decrement time_seconds, and convert it into a time string that is copied into the .time part of the msg. The raw_adc_result is stored in msg.adc, and the led2_intensity is stored in msg.intensity. The task then checks the display_mode to see what msg.cmd should be, and sets it accordingly. This message is then sent to the display_queue, for use in the display task. Then, if the time_seconds has decremented to 0, we change countdown_running to a 0, and notify the main state task that the countdown has completed with the message NOTIFY_COUNTDOWN. An else block runs when count_running is not enabled, and this block delays for a short period and obtains a new xLastWakeTime. This is to avoid the entire system from stalling when the countdown is not enabled, as this task is set at a high priority.

The final task in this module is `v_display_task()`. This task is responsible for display updates during the countdown. We first initialize a `display_msg_t` `msg` to store what was passed from the queue. Within the `for(;;)` loop of the task, we check to see if there's any messages in the `display_queue`. If yes, we take the `uart_sem`, and switch on `msg.cmd` to choose what to display. If in `DISP_TIME`, we only display the time remaining, clearing the display each time with an escape code. Otherwise, if we are in `DISP_FULL`, we clear the display, print the time remaining, ADC reading, and LED intensity. Both of these numerical values are converted to strings using `int_to_str_dec()` so we can use `Disp2Dec()`. A default case exists in case something has gone wrong. After the switch statement, we return the `uart_sem` to allow other tasks to access the shared resource.

countdown_module.h

This is the header file associated with the countdown module. It begins by including the necessary headers, and proceeds to define a macro for `NUM_ENTRY`, which is 4 in our case for four digits of entry, and for `END_TIMEOUT`, which is 5 seconds converted to ticks, and is used as the timer value for the timeout. The variables used to control the timers are made externs for use in multiple files. This includes the `time_entry` buffer with width `NUM_ENTRY`, the `time_ready` flag, `time_entry_index`, `time_seconds`, the `countdown_running` flag, and the `time_string` buffer, initialized to `NUM_ENTRY` plus space for the null terminator. The `end_timer` handle, and semaphore handles `time_sem` and `uart_sem` are also declared as externs. An enum type is created for `main_notify_t` for the notification values from the countdown task and end timeout to the main state task, and the `main_state_task_handler` is defined.

For the display variables, an enum type is created for `display_cmd_t` for use in the struct type `display_msg_t`'s `cmd` entry. `Display_msg_t` also has a time string buffer `.time`, a `uint16_t .adc`, and a `uint16_t .intensity` to hold the time to display, the adc result, and the led2 intensity. Another enum type is used for `disp_mode_t` that is used in the main state task to signify the desired mode of the display from main. A `disp_mode_t` type variable `display_mode` is set as extern, along with the handle of `display_queue`. Finally, prototypes for `v_countdown_task()`, `v_display_task()`, and `v_end_callback()` are created.

init.c

This file includes the initializer functions for the IOs and timers. It begins by including its header, and then has the function `IOinit()`. We first configure the LED pins as outputs by setting their respective `TRIS` bits to 0. We then use the `LAT` registers to ensure the LEDs are defaulted to off. All pins that can be configured to analog are set to digital (note that the pin for the ADC is set to analog in `main()`). `TRIS` bits are set to 1 for the pushbutton inputs, and pull ups are enabled, as our buttons are active low. `IOC` flag bits are cleared for each pin, the ports, and the global `IOC` flag. For our implementation, a dummy read must be done of the ports for proper

functioning of the IOC interrupts, so this is done as part of the initialization. Both positive and negative edge interrupts are enabled for each of the buttons, as the implementation requires detection both when a button is pressed and released for proper functioning. The IOC is then enabled in PADCON, a priority is set, and the global IOC interrupt is enabled. The flags are then cleared one more time for safety.

The file also includes timerInit(), which initializes both timers 2 and 3. FreeRTOS co-opts timer1, so it is not available for our use. The T32 bit is set to 0 to operate timers 2 and 3 as separate 16-bit timers as opposed to one 32-bit timer. For timer 2, we use a 1:256 prescaler, and source from the internal clock. The timer must continue operation in idle mode for proper function of the system, so this is set next. A priority is set, the flag is cleared, and the timer interrupt is enabled. Similarly, timer 3 uses a 1:256 prescaler and sources from the internal clock. It continues in idle mode, and a priority is set for the timer. The interrupt flag is cleared as well, but the timer interrupt is *not* enabled for this timer, as only the ADC is running on the timer, and it will convert automatically without an ISR.

init.h

This is the header associated with *init.c*. It includes the necessary <xc.h> header, and holds the prototypes for IOinit() and timerInit().

led_module.c

This file includes all operation involving the LEDs in the system. It begins by including its header, and proceeds to define a number of variables. For the PWM implementation, we define PWM_TON and PWM_PER to 400 to begin with. This value for the PWM period was decided upon based on experimentation to avoid flickering in the PWM signals. Flags are defined for both pwm_on and decrease, and they are defaulted to 1 for the original pulsing of LED2. Handles are defined for led_blink_timer, led_alternate_timer, and the led2_sem. Next, a few variables are defined to help with the LED2 case statement. This includes an led_mode_t type led2_mode that is defaulted to LED_MODE_OFF, uint8_t variables led2_blink_phase, led2_output_enable, and led1_blink_enable that are all defaulted to 0, and a uint16_t led2_variable_duty that is set to 1.

In the module, v_blink_callback_count() is a timer callback that is responsible for creating the blinking of LEDs 1 and 2 in the countdown state. It toggles led2_blink_phase, and directly modifies LED1 if the blinking is enabled on it.

v_blink_callback_alternating() is a timer callback responsible for creating the rapid alternating blinking of LEDs 0 and 1 in the end state. It directly toggles the state of LEDs 0 and 1 to create the blinking, and the alternating is forced when this timer is enabled in the main state task.

Finally, the timer 2 ISR is also in this file. It is responsible for a case statement for LED2 that allows us to run all of the modes of the LED, including pulsing, blinking, solidly on, and off. Upon entry, the timer2 start count is reset to 0, the interrupt flag is cleared, and a variable is created for the duty of the LED. We then switch on the mode of the LED2. In LED_MODE_OFF, the led2_output_en is turned off, and the duty is set to 0. In LED_MODE_ON, the led2_output_en is turned on, and the duty of the LED is set to the led2_variable_duty that is given from the ADC. In LED_MODE_BLINK, the duty is again set to the ADC-given led2_variable_duty. The led2_output_en is set by the phase of the led2_blink_phase, where if the blink phase is 1, the output is on. This is how we control the blinking while PWM is occurring. In LED_MODE_PULSE, the led2_output_enable is set to 1. If decrease is high, the PWM_TON is reduced by PULSE_STEP_SIZE to decrease the duty cycle of the LED and help sweep through brightness. Then, if PWM_TON is less than the PULSE_STEP_SIZE, the next step will cause overflow, we set the decrease flag to 0. If decrease is low, the PWM_TON is increased by PULSE_STEP_SIZE to increase the duty cycle and sweep up through the brightness. Then, if the PWM_TON is more than PWM_PER - PULSE_STEP_SIZE, the next step will cause the on time to exceed the total period, causing overflow later on, so the decrease flag is turned high. The duty is then set to the PWM_TON.

Upon exiting this switch statement, we check to see if the led2_output_en is on. If yes, we check if pwm_on is high. If yes, we do one phase of the pwm. LED2 is set off, the timer stop count PR2 is set to PWM_PER - duty, and the pwm_on flag is swapped. Otherwise, LED2 is turned on, and the PR2 is simply set to duty. We then swap the pwm_on flag. Otherwise, if led2_output_en is off, the output is not enabled and LED2 is forced off. This effectively implements PWM on LED2. A context switch is triggered if a higher priority task is woken.

led_module.h

This is the header file accompanying *led_module.c*. It begins by including the necessary headers, and then defines macros for the LEDs. More macros are defined for BLINK_RATE, which is defined as 1s in ticks, PULSE_STEP_SIZE, which is defined as 3 for smooth sweeping of the brightness in pulse mode, and ALT_BLINK_RATE which is 200ms in ticks for the rapid alternating rate of LEDs 0 and 1 in the end state. The PWM_TON and PWM_PER variables are declared as externs, along with the handles for led_blink_timer, led_alterate_timer, and the led2_sem. An enum type led_mode_t is defined for the modes of the LEDs, including off, on, blink, and pulse. Led2_mode is a variable of that type, and is declared extern. Variables for led2_blink_phase, led1_blink_en, led2_variable_duty, and led2_output_en are also declared extern here. Then, prototypes for v_blink_callback_count and v_blink_callback_alternating are defined as well.

uart.c

This file includes all the operations that set up UART, process data into strings for UART, and transmit and receive with UART. Functions that were given to us and were not changed are not discussed. This includes InitUART2, XmitUART2, Disp2String. The first function we'll discuss is int_to_str_dec. This function takes in a decimal number and converts it to a string so that it can be output to the terminal via UART. By using the modulo operator, the digit starting from the least significant to the most significant are stored in a string array one by one. Due to this implementation, the number would be written backwards so it is necessary to reverse the string. This is accomplished by temporarily storing the digit on the left starting from the most significant and then assigning the start of the string with the least significant digit. The least significant digit is then replaced with the old most significant digit that was stored. This effectively swaps the position of the beginning and end of the string. The next most significant and least significant digits are swapped through a similar process until the start is less than the end. The string is then ended with a null terminator before being output of the function.

The other function in uart.c that we used is UART's receive interrupt ISR. When the user inputs a character using the keyboard, the input is stored in UART's receive register. This character is then assigned to a char type variable called c. This variable is then sent to the uart_rx_queue using xQueueSendFromISR. This type of queue send needs to be used as it doesn't have the ability to block. The UART receive interrupt flag is then cleared. If during this interrupt, xHigherPriorityTaskWoken is changed to pdTRUE due to unblocking a higher priority task when it is sent to the queue, a context switch is requested. This will be executed as soon as the ISR finishes executing and will switch to the higher priority task immediately from the interrupted task after saving the context.

uart.h

This is the header associated with uart.c. It includes the necessary <xc.h> header and string.h, and holds the prototypes for InitUART2, Disp2String, XmitUART2, RecvUart, RecvUartChar, Disp2Dec, and int_to_str_dec.

Tasks

v_button_task

Description

The intended functionality of the button task is to track which buttons were pressed and for how long. There are four types of click that are tracked. EVT_CLICK and EVT_LONG for short and long presses for a single button. EVT_COMBO and EVT_LONG_COMBO for short and long presses of multiple buttons. This information is then passed to main to be processed by v_main_state_task. Within the task is an implementation for debouncing. This is accomplished by sending a semaphore from the ISR on the IOC of any button. When the semaphore is

available the task starts executing its logic. The first line of this logic is `vtaskdelay` which is used to block the task for our debounce time. After the delay finishes, the states of all 3 buttons are read and converted into a button mask. The duration that each button pressed is found by tracking the tick count when the button was pressed and when it was released using `xTaskGetTickCount`. Depending on how long the duration was, the button event is assigned as either a long or short press. The button mask, type of press, and when it was pressed is stored in a struct type and it's passed to main through a queue.

Justification

This task is the second highest priority task and it's aperiodic, executing only when the IOC interrupt ISR releases a semaphore. The task is this priority as we need the system to respond quickly to the button presses and don't want the button processing to be delayed by other tasks. The task itself also has a short execution time so it won't monopolize the CPU and starve the other tasks. The task is aperiodic as it only needs to be executed when a button is pressed.

Inter-task communication and mutex mechanisms

This task only communicates with `v_main_state_task` as it is the only task with a higher priority. The data processed within the button task is sent to the main task using a queue. The mutex mechanism used in this task is a binary semaphore. This semaphore is released when an IOC interrupt occurs and unblocks the button task. It is a binary semaphore as this semaphore is only used for this task and it's only used to signal that a button press has occurred.

`v_main_state_task`

Description

This task is used to control the state machine for our implementation. Before entering the state machine, 2 things are checked. First, if there was a button press input, which sets a button press flag high. Second, it has received a notification from `v_countdown_task` or the end timer's callback function. After these assignments, the first state `STATE_WAIT` is entered. Within this state, LED 2 pulses and if PB1 is clicked it prompts the user to input a time on the terminal. It then transitions to `STATE_INPUT`. In this state, LED 2 stops pulsing and pressing PB2 and PB3 will set the counters value used for the timer to the user's input. The user inputs are sent to the count down task through the global variable `time_seconds`. While waiting for the user to make an input, LED1 and LED2 alternate blinking. The FSM transitions to the `STATE_COUNT` when PB2 and PB3 are pressed and a valid time is input. In `STATE_COUNT`, the timer will start to count down. During this process, the user can input `i` to change what's displayed on the screen and can hit `b` to switch between the modes of led 2. The user can also click PB3 to pause the timer and long press it to abort it. When either the timer counter reaches 0 or the timer is aborted, the FSM transitions to `STATE_END`. In this state, there is a 5 second delay before it clears the screen and prompts press `pb1` to begin. The state then transitions back to the wait state.

Justification

The task structure executes periodically approximately every 1ms but the events that are processed within the task are aperiodic. This task has the third highest priority. It was decided to be at this level because while it needs to respond quickly to user inputs and system events, it doesn't need the immediate responsiveness and timing precision the countdown and button tasks needed. The task structure needs to be periodic as it is essentially replacing the main while loop and it needs to check for multiple asynchronous events so that the states can transition. It executes every 1ms so that any changes or inputs are detected and dealt with quickly.

Inter-task communication and mutex mechanisms

The task communicates with every task besides the display task. One way it does this is through queues. It receives the button press information from the button task through the `processed_button_queue`. It receives characters via UART from the `uart_rx_queue`. Another form of communication between the tasks are notifications. It receives a notification from the countdown task when the counter reaches 0. When this notification happens, the main task sets LED 2 on and turns on the end timer. When the end timer finishes, its callback function sends out another notification that transitions the FSM from `STATE_END` to `STATE_WAIT`. Additionally, the main task communicates to the countdown task through global variables. The exact way is described in the section below. Also, the main task interacts with the LED module we created through global variables. Flags and cases are set within the main task that controls the behavior of the LEDs. In terms of mutex mechanisms, there are 2 semaphores that are used. One is so that the main task has exclusive access to the UART resources when displaying certain messages. The other one is used whenever the global variable, `time_seconds`, is assigned a value in the main task. This occurs in the input state and count state, where it is set to the user input and zero, respectively. The semaphore here prevents race conditions and data corruption by stopping other tasks from accessing this variable while it is changing.

v_countdown_task

Description

The countdown task is used to process counting down the user input and preparing the current value of the timer to be displayed on the terminal. The count down task is enabled when the time left is non-zero and a flag tracking if the countdown is high. The task is then delayed for 1s using `vTaskDelayUntil` which syncs the decrementation of the timer to always occur every 1s. If during this 1s delay, the timer is paused or aborted, the system tick count at that point is saved so that the 1s delay is synced if the timer is resumed. After this check, a semaphore is used to protect the count variable while it is being decremented and processed into minutes and seconds. The minutes and seconds are then processed into a string. A struct type called `msg` holds a copy of the string as well as the raw adc value, the corresponding intensity value of LED 2, and a flag that controls whether the timer is displayed alone or with the ADC reading and LED 2 intensity.

Once all this info is copied into the struct type, it is sent to the display task through a queue. If the count variable is equal to 0, main receives a notification and it is processed accordingly. One last note is that if the count down isn't currently being executed, a 10ms task delay is executed to prevent this task from starving the lower priority tasks.

Justification

The task happens periodically every 1 second and it has the highest priority. Since the goal of this project was to create a timer, we felt that it was important to ensure that it was able to count down reliably without being interrupted or delayed by lower priority tasks. The task is periodic as this ensures the count down is executed every 1s.

Inter-task communication and mutex mechanisms

This task interacts with other tasks using queues and global variables. A copy of the struct type msg variable, holding the processed timer information, is sent to v_display_task through a queue called display_queue. There are also some global variables used like time_seconds, countdown_running, raw_duty_cycle, and led2_intensity. Time_seconds holds the time in seconds that the user input during the main task. Countdown_running is a flag toggled on and off in the main task that controls if the count down process happens within the countdown task. Raw_duty_cycle and led2_intensity were calculated and assigned in the ADC module. These global variables are assigned every time Timer3 goes off. Within this task, a mutex semaphore is used to protect processes that need to read or process time_seconds so that race conditions are prevented and the data is consistent when it is accessed.

v_display_task

Description

This task is used to display the information sent to the display_queue. It only executes when there is data within the queue. When this happens, a UART semaphore is taken and a case statement is used to display either the timer alone or the timer with the ADC reading and LED 2 intensity. The outcome of the case statement is decided by a flag set in the msg variable during the countdown task. If the flag equals 0 then it clears the terminal and displays the current timer value. If the flag equals 1 then the current timer value is displayed. Additionally, the integer adc reading and LED intensity is converted to a string before being displayed. If the flag is set to some unknown value, then an error message is displayed instead.

Justification

The task is the fourth highest priority, meaning it is the lowest priority, and is aperiodic. This task is the lowest priority because while displaying to the terminal is important, it does not have any critical timing. It also happens fast enough that it is not possible to process the delays due to being blocked by higher priority tasks. The task happens aperiodically as it should only execute

when there is something to display, which is why it gets blocked by queue receive until it receives something.

Inter-task communication and mutex mechanisms

The task communicates with the count down task through a queue. The count down task sends the data that needs to be displayed and it is received in this task through the display_queue. This task uses a mutex semaphore that protects the task when it is executing UART. This exists as there are some UART calls also in main, and so to prevent garbled text from appearing, the semaphore ensures that the task finishes using UART.

Flow diagram Notes:

The states in main task were shortened to:

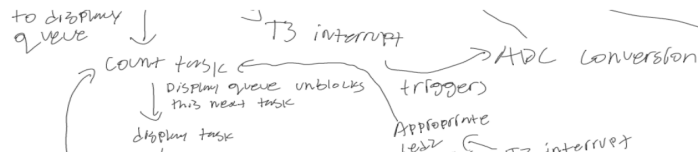
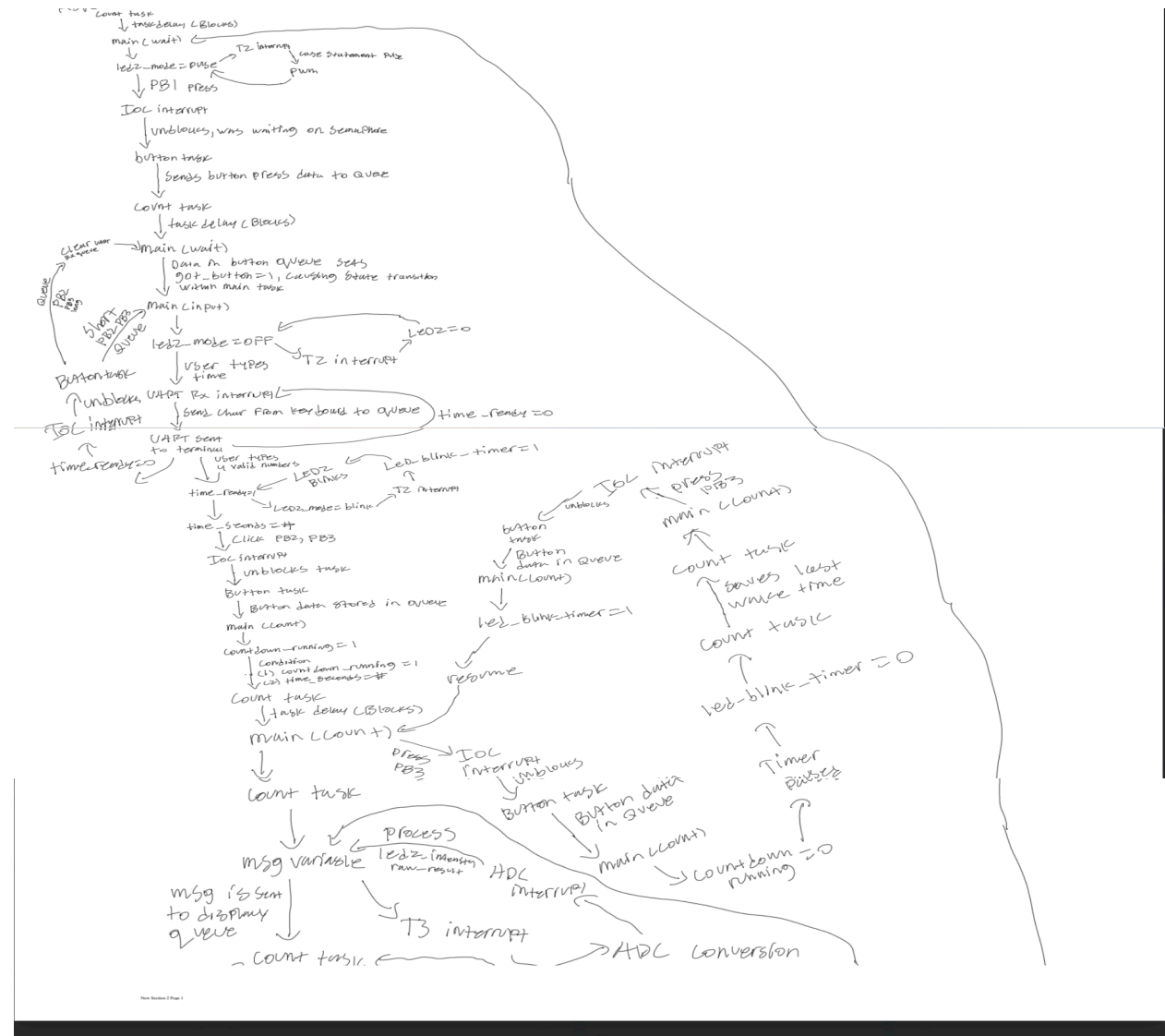
main(wait) = STATE_WAIT

main(input) = STATE_INPUT

main(count) = STATE_COUNT

main(end) = STATE_END

Flow Diagram



Flow Diagram Continued

